7CS034 Data Science

Week 1 Introduction to Data Science

Dr. Liam Naughton

1 Module Information

Welcome to **7CS034 Data Science**. This module is taken by students from M.Sc. Data Science and M.Sc. Artificial Intelligence.

- Lecturer: Dr. Liam Naughton
- email: `L.Naughton@wlv.ac.uk`

1.1 Timetable

Each week we will have

- A **lecture** - Tuesday 14:00-16:00 MC001.
- A **lab session** - Friday 10:00-13:00 - check **your** timetable for **your** room.
- A **Q&A session** - Monday 16:00-17:00 - MS Teams.

1.2 How will I be assessed?

Your assessment will consist of

- **Presentation with online submission** - 50%
- **Coursework** - 50%

More information on both will be given in week 3.

1.3 What will I need?

Notes will be provided in advance of each lecture and posted on canvas. For the labs we will use the R system for statistical computing which is freely available to download here

`www.r-project.org`

After you download and install R you should also download and install the Integrated Development Environment (IDE) RStudio from the link below.

`rstudio.com/products/rstudio/`

A video tutorial on how to do this is on Canvas. Both R and RStudio are also available via AppsAnywhere.

1.4 Access to Equipment

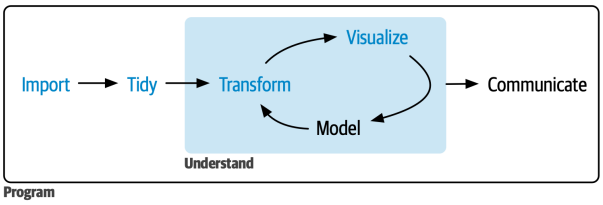
If you do not have access to a computer then you might be able to borrow one from the library on campus. Please contact them directly about this.

1.5 Course Text

Much of this module is based around the excellent free eBook **R for Data Science, 2nd Edition** by Hadley Wickham, Mine Çetinkaya-Rundel & Garrett Grolemund which can be accessed at the following link https://r4ds.hadley.nz/

2 Introduction

Our goal in this module is to give you a rapid overview of the main tools of data science: importing, tidying, transforming, and visualizing data, as shown in below. We want to show you the "whole game" of data science giving you just enough of all the major pieces so that you can tackle real datasets.



2.1 What is Data Science?

There are many definitions of data science floating around and the answer you get often depnds on the background of the person you ask. For example, a mathematician might be likely to say that "data science is mainly about statistics", or a computer scientist might be expected to say that "data science is really just artificial intelligence". Let's see what some big players in industry say.

Oracle

As a specialty, data science is young. It grew out of the fields of statistical analysis and data mining. The Data Science Journal debuted in 2002, published by the International Council for Science: Committee on Data for Science and Technology. By 2008 the title of data scientist had emerged, and the field quickly took off. There has been a shortage of data scientists ever since, even though more and more colleges and universities have started offering data science degrees.
A data scientist's duties can include developing strategies for analyzing data, preparing data for analysis, exploring, analyzing, and visualizing data, building models with data using programming languages, such as Python and R, and deploying models into applications.
The data scientist doesn't work solo. In fact, the most effective data science is done in teams. In addition to a data scientist, this team might include a business analyst who defines the problem, a data engineer who prepares the data and how it is accessed, an IT architect who oversees the underlying processes and infrastructure, and an application developer who deploys the models or outputs of the analysis into applications and products.

SAS

Data scientists are a new breed of analytical data expert who have the technical skills to solve complex problems – and the curiosity to explore what problems need to be solved. They're part mathematician, part computer scientist and part trend-spotter. And, because they straddle both the business and IT worlds, they're highly sought-after and well-paid. Who wouldn't want to be one?

SAS ctd.

They're also a sign of the times. Data scientists weren't on many radars a decade ago, but their sudden popularity reflects how businesses now think about big data. That unwieldy mass of unstructured information can no longer be ignored and forgotten. It's a virtual gold mine that helps boost revenue – as long as there's someone who digs in and unearths business insights that no one thought to look for before. Enter the data scientist. Where did they come from?
Many data scientists began their careers as statisticians or data analysts. But as big data (and big data storage and processing technologies such as Hadoop) began to grow and evolve, those roles evolved as well. Data is no longer just an afterthought for IT to handle. It's key information that requires analysis, creative curiosity, and a knack for translating high-tech ideas into new ways to turn a profit.

IBM

Data science combines math and statistics, specialized programming, advanced analytics, artificial intelligence (AI), and machine learning with specific subject matter expertise to uncover actionable insights hidden in an organization's data. These insights can be used to guide decision making and strategic planning. The accelerating volume of data sources, and subsequently data, has made data science one of the fastest growing field across every industry. As a result, it is no surprise that the role of the data scientist was dubbed the "sexiest job of the 21st century" by Harvard Business Review. Organizations are increasingly reliant on them to interpret data and provide actionable recommendations to improve business outcomes. The data science lifecycle involves various roles, tools, and processes, which enables analysts to glean actionable insights. Typically, a data science project undergoes the following stages:

- **Data ingestion:** The lifecycle begins with the data collection–both raw structured and unstructured data from all relevant sources using a variety of methods. These methods can include manual entry, web scraping, and real-time streaming data from systems and devices. Data sources can include structured data, such as customer data, along with unstructured data like log files, video, audio, pictures, the Internet of Things (IoT), social media, and more.

- **Data storage and data processing:** Since data can have different formats and structures, companies need to consider different storage systems based on the type of data that needs to be captured. Data management teams help to set standards around data storage and structure, which facilitate workflows around analytics, machine learning and deep learning models. This stage includes cleaning data, deduplicating, transforming and combining the data using ETL (extract, transform, load) jobs or other data integration technologies. This data preparation is essential for promoting data quality before loading into a data warehouse, data lake, or other repository.

IBM ctd.

- **Data analysis:** Here, data scientists conduct an exploratory data analysis to examine biases, patterns, ranges, and distributions of values within the data. This data analytics exploration drives hypothesis generation for a/b testing. It also allows analysts to determine the data's relevance for use within modeling efforts for predictive analytics, machine learning, and/or deep learning. Depending on a model's accuracy, organizations can become reliant on these insights for business decision making, allowing them to drive more scalability.

- **Communicate:** Finally, insights are presented as reports and other data visualizations that make the insights—and their impact on business—easier for business analysts and other decision-makers to understand. A data science programming language such as R or Python includes components for generating visualizations; alternately, data scientists can use dedicated visualization tools.

AWS

Data science is the study of data to extract meaningful insights for business. It is a multidisciplinary approach that combines principles and practices from the fields of mathematics, statistics, artificial intelligence, and computer engineering to analyze large amounts of data. This analysis helps data scientists to ask and answer questions like what happened, why it happened, what will happen, and what can be done with the results. Data science is important because it combines tools, methods, and technology to generate meaning from data. Modern organizations are inundated with data; there is a proliferation of devices that can automatically collect and store information. Online systems and payment portals capture more data in the fields of e-commerce, medicine, finance, and every other aspect of human life. We have text, audio, video, and image data available in vast quantities. While the term data science is not new, the meanings and connotations have changed over time. The word first appeared in the '60s as an alternative name for statistics. In the late '90s, computer science professionals formalized the term. A proposed definition for data science saw it as a separate field with three aspects: data design, collection, and analysis. It still took another decade for the term to be used outside of academia. Artificial intelligence and machine learning innovations have made data processing faster and more efficient. Industry demand has created an ecosystem of courses, degrees, and job positions within the field of data science. Because of the cross-functional skillset and expertise required, data science shows strong projected growth over the coming decades.

AWS ctd.

Data science is used to study data in four main ways:

1. **Descriptive analysis:** Descriptive analysis examines data to gain insights into what happened or what is happening in the data environment. It is characterized by data visualizations such as pie charts, bar charts, line graphs, tables, or generated narratives. For example, a flight booking service may record data like the number of tickets booked each day. Descriptive analysis will reveal booking spikes, booking slumps, and high-performing months for this service.

2. **Diagnostic analysis:** Diagnostic analysis is a deep-dive or detailed data examination to understand why something happened. It is characterized by techniques such as drill-down, data discovery, data mining, and correlations. Multiple data operations and transformations may be performed on a given data set to discover unique patterns in each of these techniques.For example, the flight service might drill down on a particularly high-performing month to better understand the booking spike. This may lead to the discovery that many customers visit a particular city to attend a monthly sporting event.

3. **Predictive analysis:**

   Predictive analysis uses historical data to make accurate forecasts about data patterns that may occur in the future. It is characterized by techniques such as machine learning, forecasting, pattern matching, and predictive modeling. In each of these techniques, computers are trained to reverse engineer causality connections in the data.For example, the flight service team might use data science to predict flight booking patterns for the coming year at the start of each year. The computer program or algorithm may look at past data and predict booking spikes for certain destinations in May. Having anticipated their customer's future travel requirements, the company could start targeted advertising for those cities from February.

4. **Prescriptive analysis:**

   Prescriptive analytics takes predictive data to the next level. It not only predicts what is likely to happen but also suggests an optimum response to that outcome. It can analyze the potential implications of different choices and recommend the best course of action. It uses graph analysis, simulation, complex event processing, neural networks, and recommendation engines from machine learning.

   Back to the flight booking example, prescriptive analysis could look at historical marketing campaigns to maximize the advantage of the upcoming booking spike. A data scientist could project booking outcomes for different levels of marketing spend on various marketing channels. These data forecasts would give the flight booking company greater confidence in their marketing decisions.

First you must **import** your data into R. This typically means that you take data stored in a file, database, or web application programming interface (API), and load it into a data frame in R. If you can't get your data into R, you can't do data science on it!

Once you have tidy data, a common first step is to **transform** it. Transformation includes narrowing in on observations of interest (like all people in one city, or all data from the last year), creating new variables that are functions of existing variables (like computing speed from distance and time), and calculating a set of summary statistics (like counts or means). Together, tidying and transforming are called **wrangling**, because getting your data in a form that's natural to work with often feels like a fight!

Once you have tidy data with the variables you need, there are two main engines of knowledge generation: visualisation and modelling. These have complementary strengths and weaknesses so any real analysis will iterate between them many times.

Visualisation is a fundamentally human activity. A good visualisation will show you things that you did not expect, or raise new questions about the data. A good visualisation might also hint that you're asking the wrong question, or you need to collect different data. Visualisations can surprise you, but don't scale particularly well because they require a human to interpret them.

Models are complementary tools to visualisation. Once you have made your questions sufficiently precise, you can use a model to answer them. Models are a fundamentally mathematical or computational tool, so they generally scale well. Even when they don't, it's usually cheaper to buy more computers than it is to buy more brains! But every model makes assumptions, and by its very nature a model cannot question its own assumptions. That means a model cannot fundamentally surprise you.

The last step of data science is **communication**, an absolutely critical part of any data analysis project. It doesn't matter how well your models and visualisation have led you to understand the data unless you can also communicate your results to others.

Surrounding all these tools is **programming**. Programming is a cross-cutting tool that you use in every part of the project. You don't need to be an expert programmer to be a data scientist, but learning more about programming pays off because becoming a better programmer allows you to automate common tasks, and solve new problems with greater ease.

3 What we won't do

3.1 Big data

We'll focus on small, in-memory datasets. This is the right place to start because you can't tackle big data unless you have experience with small data. The tools you learn will easily handle hundreds of megabytes of data, and with a little care you can typically use them to work with 1-2 Gb of data.

If your data is bigger than this, carefully consider if your big data problem might actually be a small data problem in disguise. While the complete data might be big, often the data needed to answer a specific question is small. You might be able to find a subset, subsample, or summary that fits in memory and still allows you to answer the question that you're interested in. The challenge here is finding the right small data, which often requires a lot of iteration.

Another possibility is that your big data problem is actually a large number of small data problems. Each individual problem might fit in memory, but you have millions of them. For example, you might want to fit a model to each person in your dataset. That would be trivial if you had just 10 or 100 people, but instead you have a million. Fortunately each problem is independent of the others (a setup that is sometimes called embarrassingly parallel), so you just need a system (like Hadoop or Spark) that allows you to send different datasets to different computers for processing. Once you've figured out how to answer the question for a single subset using the tools described in this module, you learn new tools like sparklyr, rhipe, and ddr to solve it for the full dataset.

3.2 Python, Julia, and friends

In this module, you won't learn anything about Python, Julia, or any other programming language useful for data science. This isn't because we think these tools are bad. They're not! And in practice, most data science teams use a mix of languages, often at least R and Python.

However, we strongly believe that it's best to master one tool at a time. You will get better faster if you dive deep, rather than spreading yourself thinly over many topics. This doesn't mean you should only know one thing, just that you'll generally learn faster if you stick to one thing at a time. You should strive to learn new things throughout your career, but make sure your understanding is solid before you move on to the next interesting thing.

We think R is a great place to start your data science journey because it is an environment designed from the ground up to support data science. R is not just a programming language, but it is also an interactive environment for doing data science. To support interaction, R is a much more flexible language than many of its peers. This flexibility comes with its downsides, but the big upside is how easy it is to evolve tailored grammars for specific parts of the data science process. These mini languages help you think about problems as a data scientist, while supporting fluent interaction between your brain and the computer.

3.3 Non-rectangular data

This module focuses exclusively on rectangular data: collections of values that are each associated with a variable and an observation. There are lots of datasets that do not naturally fit in this paradigm, including images, sounds, trees, and text. But rectangular data frames are extremely common in science and industry, and we believe that they are a great place to start your data science journey.

3.4 Statistics

You'll learn this in 7CS039.

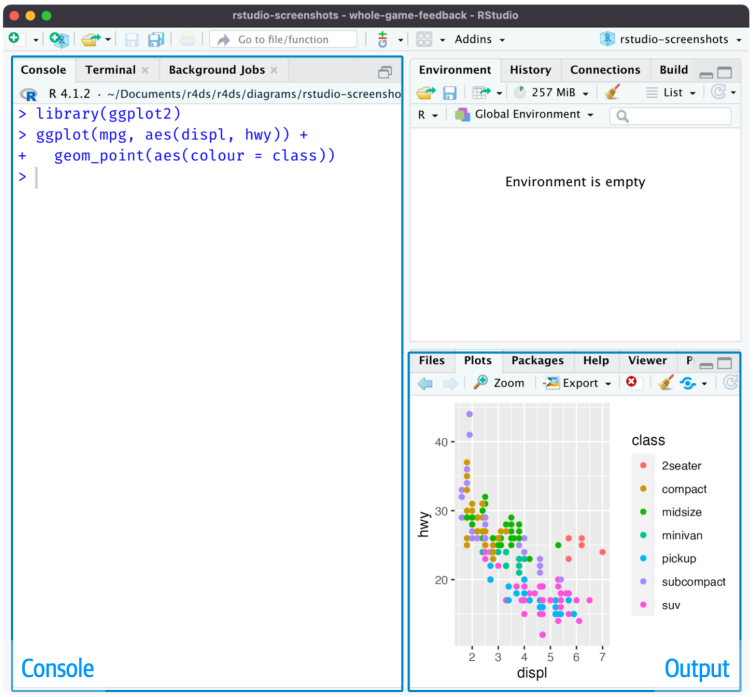
4 Prerequisites

4.1 R - the base system

Go to https://cloud.r-project.org/ and download the version which is right for your system.

4.2 RStudio

This is the Integrated Development Environment (IDE) we'll use for the labs/workshops. Go here https://posit.co/download/rstudio-desktop/ and download the version which is right for your system. When you start RStudio, you'll see two key regions in the interface: the console pane and the output pane. For now, all you need to know is that you type the R code in the console pane and press enter to run it. You'll learn more as we go along!



4.3 The Tidyverse

You'll also need to install some R packages. An R package is a collection of functions, data, and documentation that extends the capabilities of base R. Using packages is key to the successful use of R. The majority of the packages that you will learn in this module are part of the so-called tidyverse. All packages in the tidyverse share a common philosophy of data and R programming and are designed to work together. You can install the complete tidyverse with a single line of code:

```
install.packages("tidyverse")
```

On your computer, type that line of code in the console, and then press enter to run it. R will download the packages from CRAN and install them on your computer. You will not be able to use the functions, objects, or help files in a package until you load it

with `library()`. Once you have installed a package, you can load it using the `library()` function:

```
library(tidyverse)
#> ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
#> ✔ dplyr     1.1.3     ✔ readr     2.1.4
#> ✔ forcats   1.0.0     ✔ stringr   1.5.0
#> ✔ ggplot2   3.4.3     ✔ tibble    3.2.1
#> ✔ lubridate 1.9.3     ✔ tidyr     1.3.0
#> ✔ purrr     1.0.2
#> ── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
#> ✖ dplyr::filter() masks stats::filter()
#> ✖ dplyr::lag()    masks stats::lag()
#> ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
```

This tells you that tidyverse loads nine packages: `dplyr`, `forcats`, `ggplot2`, `lubridate`, `purrr`, `readr`, `stringr`, `tibble`, `tidyr`. These are considered the core of the tidyverse because you'll use them in almost every analysis. Packages in the tidyverse change fairly frequently. You can see if updates are available by running `tidyverse_update()`.

5 Other Packages

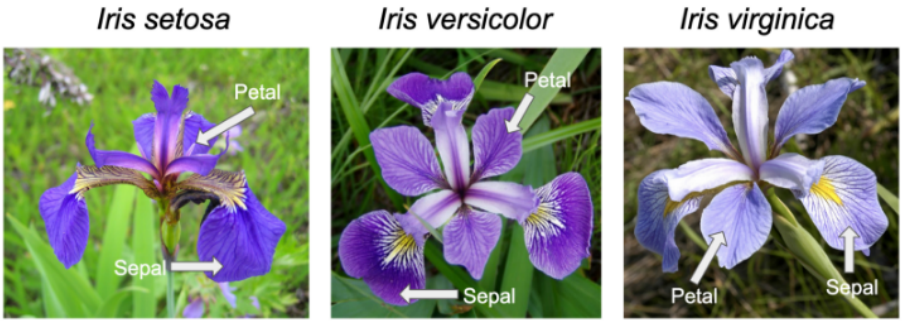
There are more than 19,000 packages available for `R`. That's a lot. We'll use many packages from outside the tidyverse in this module. For example, we'll use the following packages because they provide interesting datasets for us to work with in the process of learning R:

```
install.packages(
  c("arrow", "babynames", "curl", "duckdb", "gapminder",
    "ggrepel", "ggridges", "ggthemes", "hexbin", "janitor", "Lahman",
    "leaflet", "maps", "nycflights13", "openxlsx", "palmerpenguins",
    "repurrrsive", "tidymodels", "writexl")
  )
```

6 Exploring the Iris Dataset

In this section, we will provide a quick introduction to `R` using the famous `iris` dataset. We will cover loading data, exploring the dataset, basic data manipulation, and plotting. By the end, you should have a good understanding of how to get started with `R`.

The `iris` dataset is a classic and widely used dataset in the field of data science and machine learning. The dataset consists of 150 samples from three species of iris flowers: Iris setosa, Iris versicolor, and Iris virginica. Each sample contains four features, which are the lengths and widths of the sepals and petals (in centimeters). The dataset has become a popular choice for testing and demonstrating various data science and machine learning techniques due to its simplicity and well-defined structure. The iris dataset comes pre-loaded in `R`, so no need to import any external files.



To get an overview of the dataset, use the `head()` and `summary()` functions:

```
head(iris)
```

```
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
>
```

To quickly print some summary statistics we can do

```
summary(iris)
```

```
> summary(iris)
  Sepal.Length    Sepal.Width     Petal.Length    Petal.Width          Species
 Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100   setosa    :50
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300   versicolor:50
 Median :5.800   Median :3.000   Median :4.350   Median :1.300   virginica :50
 Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
>
```

The `summary()` function in `R` provides a quick overview of the main statistical measures for numerical data. Here are short descriptions of each measure:

- Minimum (Min.): The smallest value in the dataset.
- 1st Quartile (1st Qu.): The value that separates the lowest 25% of the data from the remaining 75%; also known as the 25th percentile.
- Median (2nd Qu.): The middle value that separates the lower and upper halves of the data; also known as the 50th percentile.

- Mean: The arithmetic average of the data values, calculated by adding up all the values and dividing by the total number of values.
- 3rd Quartile (3rd Qu.): The value that separates the lowest 75% of the data from the highest 25%; also known as the 75th percentile.
- Maximum (Max.): The largest value in the dataset.

These statistical measures give you a quick snapshot of the central tendency, dispersion, and overall distribution of your numerical data.

6.1 Subsetting the data

To select specific columns in the dataset, use the $ operator or the square brackets. To select the column corresponding to petal width:

```
 Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
> iris$Petal.Width
  [1] 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 0.2 0.2 0.1 0.1 0.2 0.4 0.4 0.3 0.3 0.3 0.2 0.4 0.2 0.5 0.2 0.2 0.4 0.2 0.2
 [30] 0.2 0.2 0.4 0.1 0.2 0.2 0.2 0.2 0.1 0.2 0.2 0.3 0.3 0.2 0.6 0.4 0.3 0.2 0.2 0.2 0.2 1.4 1.5 1.5 1.3 1.5 1.3 1.6 1.0
 [59] 1.3 1.4 1.0 1.5 1.0 1.4 1.3 1.4 1.5 1.0 1.5 1.1 1.8 1.3 1.5 1.2 1.3 1.4 1.4 1.7 1.5 1.0 1.1 1.0 1.2 1.6 1.5 1.6 1.5
 [88] 1.3 1.3 1.3 1.2 1.4 1.2 1.0 1.3 1.2 1.3 1.3 1.1 1.3 2.5 1.9 2.1 1.8 2.2 2.1 1.7 1.8 1.8 2.5 2.0 1.9 2.1 2.0 2.4 2.3
[117] 1.8 2.2 2.3 1.5 2.3 2.0 2.0 1.8 2.1 1.8 1.8 1.8 2.1 1.6 1.9 2.0 2.2 1.5 1.4 2.3 2.4 1.8 1.8 2.1 2.4 2.3 1.9 2.3 2.5
[146] 2.3 1.9 2.0 2.3 1.8
>
```

To select the first three columns we do:

```
> iris[ ,1:3]
    Sepal.Length Sepal.Width Petal.Length
1            5.1         3.5          1.4
2            4.9         3.0          1.4
3            4.7         3.2          1.3
4            4.6         3.1          1.5
5            5.0         3.6          1.4
6            5.4         3.9          1.7
7            4.6         3.4          1.4
8            5.0         3.4          1.5
9            4.4         2.9          1.4
10           4.9         3.1          1.5
11           5.4         3.7          1.5
12           4.8         3.4          1.6
13           4.8         3.0          1.4
14           4.3         3.0          1.1
15           5.8         4.0          1.2
16           5.7         4.4          1.5
17           5.4         3.9          1.3
18           5.1         3.5          1.4
```

6.2 Filtering the dataset

To filter the dataset based on a condition, use the `subset()` function:

```
> subset(iris, Species == "setosa")
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1         3.5          1.4         0.2  setosa
2           4.9         3.0          1.4         0.2  setosa
3           4.7         3.2          1.3         0.2  setosa
4           4.6         3.1          1.5         0.2  setosa
5           5.0         3.6          1.4         0.2  setosa
6           5.4         3.9          1.7         0.4  setosa
7           4.6         3.4          1.4         0.3  setosa
8           5.0         3.4          1.5         0.2  setosa
9           4.4         2.9          1.4         0.2  setosa
10          4.9         3.1          1.5         0.1  setosa
11          5.4         3.7          1.5         0.2  setosa
12          4.8         3.4          1.6         0.2  setosa
13          4.8         3.0          1.4         0.1  setosa
14          4.3         3.0          1.1         0.1  setosa
15          5.8         4.0          1.2         0.2  setosa
```

6.3 Sorting the dataset

To sort the dataset by a specific column, use the `order()` function. We can combine that with selecting only certain columns:

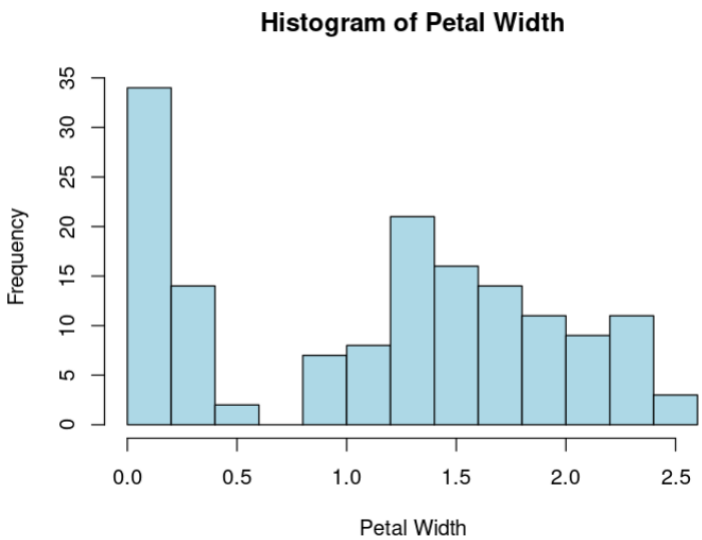
```
> iris[order(iris$Petal.Width), c("Petal.Width", "Species")]
   Petal.Width    Species
10         0.1     setosa
13         0.1     setosa
14         0.1     setosa
33         0.1     setosa
38         0.1     setosa
1          0.2     setosa
2          0.2     setosa
3          0.2     setosa
4          0.2     setosa
5          0.2     setosa
8          0.2     setosa
9          0.2     setosa
11         0.2     setosa
```

On yout screen you should see that petal width is pretty good at separating the different species. We will corroborate this with some basic plotting.

6.4 Histogram

A histogram is a graphical representation of the distribution of a dataset, where data is divided into a set of intervals or bins. The data is represented as vertical bars, with the height of each bar corresponding to the number of data points that fall within a particular bin. Histograms are used to visualize the underlying frequency distribution of a continuous variable, allowing one to identify patterns such as skewness, central tendency, and dispersion. You should be familiar with these from the pre course induction in maths & programming!?! Here we do:

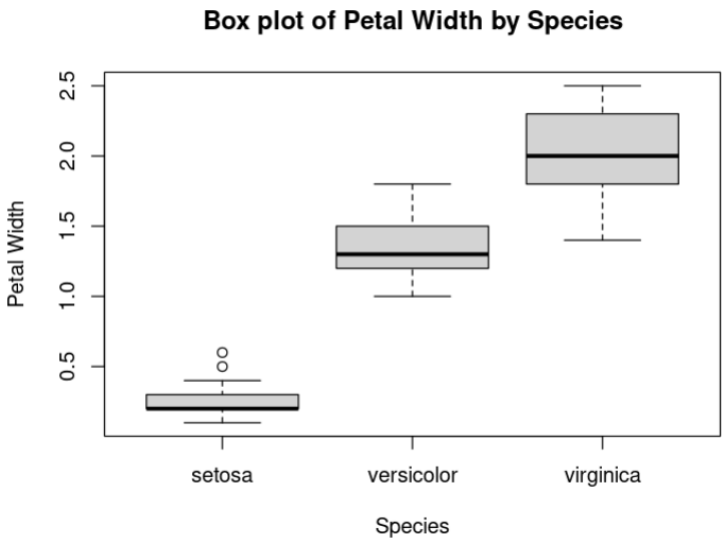
```
hist(iris$Petal.Width, main = "Histogram of Petal Width",
xlab = "Petal Width", ylab = "Frequency", col = "lightblue",
border = "black")
```



6.5 Box plot

A box plot, also known as a box-and-whisker plot, is a graphical representation of the distribution and spread of a dataset. It displays five key statistics: the minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. The "box" represents the interquartile range (IQR), which contains the middle 50% of the data, while the "whiskers" extend from the box to the minimum and maximum values. Outliers, if present, are typically represented as individual points outside the whiskers. You should be familiar with these from the pre course induction in maths & programming!?!

```
boxplot(Petal.Width ~ Species, data = iris, main =
"Box plot of Petal Width by Species", xlab = "Species",
ylab = "Petal Width")
```

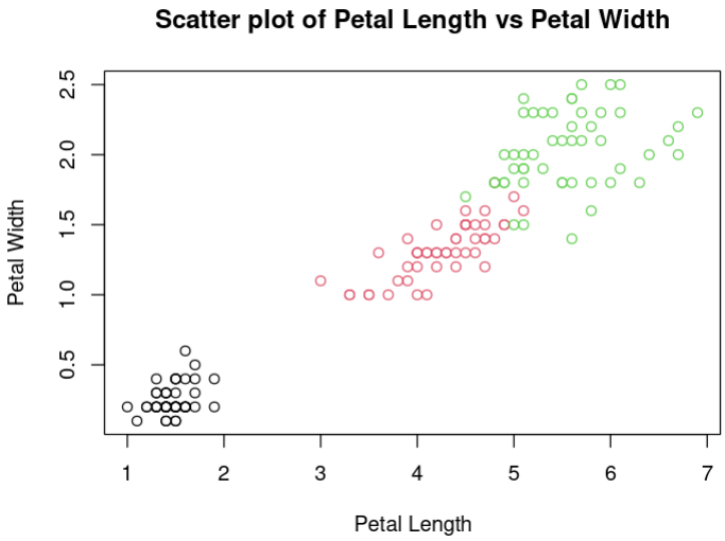


Here, we can very clearly see that petal width indeed separates the different species well. In the following plot, another variable, petal length, is included and the three species are colour-coded.

6.6 Scatter plot

A scatter plot is a graphical representation of the relationship between two variables, where each data point is represented by a dot on a two-dimensional plane. The horizontal axis (x-axis) represents one variable, while the vertical axis (y-axis) represents the other variable. By analyzing the distribution and pattern of the dots, one can determine the correlation, trends, or outliers between the two variables. You should be familiar with these from the pre course induction in maths & programming!?!

```
plot(iris$Petal.Length, iris$Petal.Width, main =
"Scatter plot of Petal Length vs Petal Width",
xlab = "Petal Length", ylab = "Petal Width",
col = iris$Species)
```



And there you have it. We've learned the basics of `R` using the iris dataset. We covered loading data, data manipulation, and basic plotting. As you continue to explore `R`, you will discover its vast capabilities and potential for analyzing and visualizing complex data.